UNIVERSIDAD TÉCNICA NACIONAL

Sede Guanacaste

Recinto: Cañas

Pruebas API testing

Curso:

Calidad y Pruebas de Software.

Aprendientes:

Christopher Ruiz Calderón

Emanuel Campos Bolandi

Diego Rodríguez Ramos

Facilitador:

Silvia Elena Alvarado Alvarado

I Cuatrimestre, 2026

Índice

1. **Introducción.** 4
2. **Objetivos:** 5
   - 2.1. **Objetivo General.** 5
   - 2.2. **Objetivos Específicos:** 5
3. **Documentos de Referencia** 6
4. **Roles y Responsabilidades.** 7
5. **Tareas.** 8
6. **Alcance.** 10
   - 6.1. **Dentro del alcance:** 10
   - 6.2. **Fuera del alcance:** 11
7. **Estrategia de pruebas.** 12
8. **Requerimientos del ambiente.** 13
   - 8.1. **Hardware:** 13
   - 8.2. **Software:** 14
   - 8.3. **Red:** 14
9. **Reporte de problemas y acciones correctivas.** 14
   - 9.1. **Flujo para reporte de defectos:** 14
     - ***Paso 1: Detección del defecto*** 14
     - ***Paso 2: Registro inicial*** 15
     - ***Paso 3: Validación del defecto*** 15
     - ***Paso 4: Decisión sobre la validez*** 15
     - ***Paso 5: Clasificación del defecto*** 15
     - ***Paso 6: Registro en el repositorio de defectos*** 16
     - ***Paso 7: Análisis y plan de corrección*** 16
     - ***Paso 8: Corrección*** 16
     - ***Paso 9: Pruebas de verificación (regresión)*** 16
     - ***Paso 10: Decisión sobre la corrección*** 16
     - ***Paso 11: Cierre del defecto*** 16
   - 9.2. **Formato de reporte de defectos:** 18
   - 9.3. **Escalamiento de problemas y Criterios de aceptación:** 19
10. **Gestión del riesgo.** 20
11. **Herramientas.** 21

12. Métricas. 22
12.1. Métricas a evaluar: 22
12.2. Plantilla: 23
13. Referencias 24

Tablas

Tabla 1 Documentos de referencia 6
Tabla 2 Roles y Responsabilidades del proyecto 7
Tabla 3 Tareas del proyecto 8
Tabla 4 Estrategia de pruebas 12
Tabla 5 Hardware 13
Tabla 6 Software 14
Tabla 7 Red 14
Tabla 8 Formato de reporte de defectos 18
Tabla 9 Escalamiento de problemas 19
Tabla 10 Criterios de aceptación 19
Tabla 11 Matriz de Riesgos 20
Tabla 12 Herramientas 21
Tabla 13 Métricas 22
Tabla 14 Plantilla 23

1. Introducción.

El sistema seleccionado para el proceso de pruebas de API es RESTful-Booker, una aplicación de código abierto diseñada específicamente para prácticas de testing de APIs. Esta aplicación simula un sistema de reservas de hotel, exponiendo una API REST que permite realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre reservas.

Principales funciones a alto nivel:

- **Creación de reservas:** Permite registrar nuevas reservas con datos de huésped, fechas y precios.
- **Consulta de reservas:** Permite obtener listados de reservas o filtrar por ID, nombre, fecha, etc.
- **Actualización de reservas:** Permite modificar total o parcialmente los datos de una reserva existente.
- **Eliminación de reservas:** Permite eliminar reservas del sistema.
- **Autenticación:** Proporciona un mecanismo de generación de tokens para operaciones que requieren autorización.

La API será sometida a un proceso sistemático de pruebas para evaluar su calidad en términos de funcionalidad, confiabilidad, rendimiento y seguridad, utilizando métricas cuantitativas y herramientas especializadas en API testing.

2. Objetivos:

2.1. Objetivo General.

Evaluar la calidad de la API RESTful-Booker mediante la aplicación de pruebas funcionales, de integración, seguridad y rendimiento, utilizando métricas y herramientas especializadas, para determinar su grado de confiabilidad y adherencia a los estándares de calidad de software.

2.2. Objetivos Específicos:

1. Analizar la documentación técnica de la API RESTful-Booker para identificar sus endpoints, parámetros, estructuras de respuesta y posibles escenarios de prueba.
2. Verificar que todos los endpoints de la API responden correctamente según la documentación oficial, validando códigos de estado HTTP y estructura de respuestas.
3. Validar el correcto funcionamiento de las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre el recurso principal "booking".
4. Comprobar que el mecanismo de autenticación funciona adecuadamente y restringe el acceso a operaciones sensibles.
5. Evaluar el comportamiento de la API ante escenarios negativos (datos inválidos, campos faltantes, IDs inexistentes).
6. Medir los tiempos de respuesta de los principales endpoints bajo condiciones normales de operación.
7. Identificar y documentar los defectos encontrados durante el ciclo de pruebas, clasificándolos por severidad y prioridad.
8. Generar métricas de calidad que permitan determinar el estado actual de la API (tasa de éxito, densidad de defectos, cobertura de endpoints).

3. Documentos de Referencia

En esta sección se presentan los documentos, estándares y guías técnicas que sirven como base para el desarrollo del presente plan de pruebas. Estas fuentes proporcionan lineamientos y buenas prácticas reconocidas en la industria del software para la evaluación de APIs, la gestión de defectos y la medición de la calidad del producto. Asimismo, permiten garantizar que las actividades de prueba se realicen siguiendo estándares internacionales y documentación oficial del sistema evaluado.

Tabla 1 Documentos de referencia

ID	Documento	Fuente	Fecha
DR-01	Documentación Oficial de RESTful-Booker	RESTful-Booker	2024
DR-02	Estándar ISO/IEC 25010:2011 - Calidad del Producto de Software	Organización Internacional de Normalización	2011
DR-03	RFC 2616 - Hypertext Transfer Protocol (HTTP/1.1)	IETF	1999
DR-04	Guía de Buenas Prácticas para API REST	Microsoft API Guidelines	2023
DR-05	Formato de Reporte de Defectos (Basado en IEEE 829)	IEEE	2008

Nota: Elaboración propia.

4. Roles y Responsabilidades.

En esta sección se definen los roles involucrados en el proceso de pruebas y las responsabilidades asignadas a cada integrante del equipo. La correcta distribución de funciones permite organizar las actividades de testing, garantizar una adecuada ejecución de las pruebas y facilitar la coordinación entre los miembros del proyecto.

Tabla 2 Roles y Responsabilidades del proyecto

Rol	Nombre	Responsabilidades
Product Owner	RESTful-Booker	- Define criterios de aceptación - Aprueba el plan de pruebas - Valida que los objetivos cumplen con lo esperado - Toma decisiones sobre la calidad aceptable
QA Lead	Christopher Ruiz Calderón	- Coordina el equipo de testing - Revisa y aprueba casos de prueba - Supervisa la ejecución de pruebas - Genera informes de resultados - Gestiona los riesgos del proceso
QA Tester	Diego Rodríguez Ramos	- Diseña casos de prueba - Configura el ambiente de pruebas - Ejecuta pruebas manuales y automatizadas - Documenta defectos encontrados - Realiza seguimiento a correcciones
QA de Automatización	Emmanuel Campos Bolandi	- Implementa scripts de prueba automatizados - Configura colecciones en Postman - Ejecuta pruebas de regresión - Genera reportes automatizados
Soporte	Equipo	- Gestiona herramientas y repositorios - Documenta la configuración del ambiente - Apoya en la resolución de problemas técnicos

Nota: Elaboración propia basada en el modelo de gestión de roles de pruebas propuesto por el International Software Testing Qualifications Board (ISTQB).

5. Tareas.

En esta sección se describen las principales actividades necesarias para llevar a cabo el proceso de pruebas de la API. Cada tarea representa una etapa del ciclo de pruebas, desde el análisis inicial de la documentación hasta la elaboración del informe final de resultados.

Tabla 3 Tareas del proyecto

ID	Tarea	Descripción	Entradas	Salidas	Responsable
T01	Análisis de documentación	Revisar la documentación oficial de la API para entender endpoints, parámetros y respuestas	Documentación API (DR-01)	Inventario de endpoints a probar	Todo el equipo
T02	Diseño de casos de prueba	Elaborar casos de prueba funcionales y no funcionales	Inventario de endpoints, Objetivos	Especificación de casos de prueba	QA Tester
T03	Configuración del ambiente	Preparar Postman/Insomnia y verificar conectividad con la API	Documentación técnica	Ambiente configurado y validado	QA de Automatización
T04	Estimación de esfuerzo	Calcular tiempo requerido para ejecutar todas las pruebas	Casos de prueba	Cronograma de ejecución	QA Lead
T05	Preparación de datos de prueba	Definir y crear los datos necesarios para las pruebas	Casos de prueba	Conjunto de datos de prueba	QA de Automatización
T06	Configuración de herramientas de reporte	Establecer plantillas para reporte de defectos e informes	Formato IEEE 829 (DR-05)	Plantillas definidas	QA Lead
T07	Revisión del plan	Validar que el plan cumple con los objetivos del curso	Borrador del plan	Plan de pruebas aprobado	Product Owner

T08	Ejecución de pruebas	Probar un subconjunto de casos para validar el ambiente	Casos de prueba, Ambiente	Resultados preliminares	Todo el equipo
T09	Análisis de resultados	Revisar los resultados obtenidos, comparar con criterios de éxito, identificar tendencias.	Resultados de ejecución (T08), Criterios de aceptación	Listado de defectos validados, métricas calculadas (tasas de éxito, densidad de defectos, tiempos), lecciones preliminares	QA Lead (con apoyo de QA Tester)
T10	Generación de informe final	Elaborar un documento con métricas, defectos encontrados, lecciones aprendidas y recomendaciones.	Resultados analizados (T09), Reportes de defectos (formato Tabla 8), Plantillas (T06)	Informe final de pruebas (PDF) que incluya: resumen ejecutivo, métricas, gráficos, defectos abiertos/cerrados, conclusiones	QA Tester (revisado por QA Lead)

Nota: Elaboración propia basada en las fases del proceso de pruebas definidas en el estándar de documentación de pruebas del Institute of Electrical and Electronics Engineers (IEEE 829).

6. Alcance.

6.1. Dentro del alcance:

El presente plan de pruebas de API se enfocará en la evaluación exhaustiva de la API RESTful-Booker, abarcando todos los endpoints documentados oficialmente. Se realizarán pruebas funcionales sobre el endpoint de autenticación (/auth) para verificar la correcta generación de tokens de acceso, elemento fundamental para las operaciones que requieren autorización.

En cuanto al recurso principal de reservas, se probará el endpoint de creación (/booking mediante método POST) para validar que las reservas se crean correctamente con todos los campos requeridos, así como el comportamiento ante variaciones en los datos de entrada. Para la consulta de información, se evaluará el endpoint de listado (/booking con método GET) en sus dos modalidades: obtención de todas las reservas y filtrado por parámetros como nombre, apellido o fechas de check-in y check-out. Complementariamente, se probará la consulta por ID específico (/booking/{id} mediante GET) para verificar la recuperación de reservas individuales.

Las operaciones de modificación serán evaluadas a través del endpoint de actualización total (/booking/{id} con método PUT) y el de actualización parcial (/booking/{id} con método PATCH), verificando que los cambios se apliquen correctamente según el tipo de modificación solicitada. Finalmente, se probará la eliminación de reservas mediante el endpoint (/booking/{id} con método DELETE) para confirmar que las reservas pueden ser removidas del sistema adecuadamente.

Para todas las operaciones mencionadas, se realizará una validación exhaustiva de los códigos de estado HTTP retornados, verificando que correspondan con lo esperado según la operación realizada (200 para éxito, 201 para creación, 400 para errores de solicitud, 401 y 403 para problemas de autenticación y autorización, 404 para recursos no encontrados, entre otros). Adicionalmente, se validará la estructura de las respuestas JSON mediante pruebas de contrato que aseguren que los campos, tipos de datos y formatos coinciden con lo especificado en la documentación.

Las pruebas negativas constituirán una parte fundamental del alcance, incluyendo escenarios como envío de datos inválidos, campos obligatorios faltantes, formatos incorrectos en fechas y valores, así como peticiones a recursos inexistentes. Estas pruebas permitirán evaluar la robustez de la API y su capacidad para manejar situaciones de error de manera adecuada.

6.2. Fuera del alcance:

Quedan excluidas del presente plan aquellas pruebas que no corresponden a la naturaleza de una API o que, por limitaciones de tiempo y recursos del proyecto académico, no podrán ser ejecutadas con la profundidad requerida. En primer lugar, no se realizarán pruebas de interfaz de usuario, dado que RESTful-Booker es exclusivamente una API y no posee una interfaz gráfica asociada que pueda ser evaluada.

Tampoco se incluirán pruebas de compatibilidad con navegadores web, ya que las interacciones con la API se realizan mediante peticiones HTTP y no a través de un navegador. Por la misma razón, quedan fuera las pruebas de experiencia de usuario (UX) y accesibilidad, propias de aplicaciones con interfaz visual.

En cuanto a pruebas no funcionales, si bien se medirán tiempos de respuesta básicos, no se realizarán pruebas de carga masiva que simulen más de cien usuarios concurrentes, debido a las limitaciones de infraestructura y al tiempo disponible para la ejecución del proyecto. Tampoco se ejecutarán pruebas de seguridad avanzadas como análisis de penetración (penetration testing), inyección SQL profunda o pruebas de denegación de servicio, las cuales requieren conocimientos especializados y herramientas más complejas.

El análisis de código fuente o pruebas de caja blanca quedan fuera del alcance, ya que el enfoque del proyecto es exclusivamente sobre pruebas de caja negra a nivel de API. Asimismo, no se evaluarán endpoints no documentados oficialmente, ni se probarán integraciones con sistemas externos como pasarelas de pago o servicios de inventario, dado que la API opera de manera independiente.

Finalmente, no se realizarán pruebas de regresión completas después de cada corrección, limitándose a pruebas puntuales de verificación sobre los defectos encontrados, en concordancia con el alcance académico del proyecto.

7. Estrategia de pruebas.

En esta sección se define la estrategia que se seguirá para evaluar el funcionamiento y la calidad de la API durante el proceso de pruebas. La estrategia establece los diferentes tipos de pruebas que se ejecutarán, los objetivos de cada una y los criterios que permitirán determinar si los resultados obtenidos son satisfactorios. De esta manera, se busca asegurar una evaluación sistemática que permita identificar posibles defectos y validar el correcto comportamiento del sistema.

Tabla 4 Estrategia de pruebas

Orden	Tipo de Prueba	Objetivo	Técnica	Criterio de Éxito
1	**Pruebas de Humo (Smoke Testing)**	Verificar que la API está operativa y responde a peticiones básicas	Ejecutar 1 petición exitosa a cada endpoint principal	100% de endpoints responden con código 2xx
2	**Pruebas Funcionales**	Validar que cada endpoint cumple con lo especificado en la documentación	Ejecutar casos de prueba para operaciones CRUD	95% de casos funcionales exitosos
3	**Pruebas de Contrato**	Verificar que las respuestas cumplen con la estructura JSON esperada	Validación de esquemas con herramientas como Ajv	100% de respuestas válidas estructuralmente
4	**Pruebas Negativas**	Evaluar cómo maneja la API escenarios de error y datos inválidos	Envío de payloads incorrectos, IDs inválidos, autenticación faltante	Respuestas con códigos de error apropiados (4xx)

5	**Pruebas de Seguridad Básica**	Validar que endpoints protegidos requieren autenticación	Intentar acceder a PUT/PATCH/DELETE sin token	Respuesta 403/401 para peticiones no autorizadas
6	**Pruebas de Integración**	Verificar flujos completos (crear → consultar → actualizar → eliminar)	Secuencias de peticiones encadenadas	Flujo completo exitoso
7	**Pruebas de Rendimiento Básico**	Medir tiempos de respuesta bajo condiciones normales	10 ejecuciones por endpoint, calcular promedio	Tiempo promedio < 1000ms

Nota: Elaboración propia basada en las estrategias de pruebas descritas en el syllabus del International Software Testing Qualifications Board.

8. Requerimientos del ambiente.

En esta sección se describen los recursos tecnológicos necesarios para llevar a cabo la ejecución de las pruebas del sistema. Se incluyen los requerimientos mínimos y recomendados de hardware, software y red, con el fin de garantizar que el ambiente de pruebas cuente con las condiciones adecuadas para ejecutar las herramientas y acceder correctamente a la API.

8.1. Hardware:

Tabla 5 Hardware

Componente	Especificación Mínima	Recomendada
Procesador	Intel Core i3 o equivalente	Intel Core i5 o superior
Memoria RAM	4 GB	8 GB o más
Almacenamiento	10 GB libres	20 GB libres
Conexión a Internet	5 Mbps	10 Mbps o superior

8.2. Software:

Tabla 6 Software

Categoría	Especificación
Sistema Operativo	Windows 10/11, macOS 12+, Ubuntu 20.04+
Navegadores (para documentación)	Chrome 120+, Firefox 115+, Edge 120+
Herramienta principal de pruebas	Postman v11+ / Insomnia 2023.5+
Cliente de terminal	Command Prompt, PowerShell, Terminal (Linux/macOS)
Editor de texto/JSON	VS Code, Sublime Text, Notepad++

8.3. Red:

Tabla 7 Red

Requisito	Descripción
URL de la API	https://restful-booker.herokuapp.com
Puertos requeridos	443 (HTTPS)
Protocolos	HTTP/1.1, HTTPS/TLS 1.2+
Proxy	No requiere configuración especial

Nota: Elaboración propia basada en buenas prácticas para la definición de ambientes de prueba descritas por el International Organization for Standardization en estándares de calidad de software.

9. Reporte de problemas y acciones correctivas.

En esta sección se describe el proceso que se seguirá para identificar, registrar, analizar y dar seguimiento a los defectos encontrados durante la ejecución de las pruebas. Este procedimiento permite asegurar que cada problema detectado sea documentado de forma estructurada, facilitando su análisis y eventual corrección. Además, el flujo de gestión de defectos contribuye a mantener la trazabilidad y el control de calidad del sistema durante todo el ciclo de pruebas.

9.1. Flujo para reporte de defectos:

Paso 1: Detección del defecto

El tester, durante la ejecución de las pruebas, identifica un comportamiento que no coincide con lo esperado según la documentación de la API. Esto puede incluir

códigos de estado incorrectos, datos erróneos en las respuestas, fallos de autenticación, entre otros.

Paso 2: Registro inicial

El tester completa el formato de reporte de defecto con toda la información relevante: título descriptivo, endpoint afectado, método HTTP utilizado, payload enviado, respuesta obtenida, respuesta esperada, pasos para reproducir, fecha y hora, y evidencia (capturas de pantalla o archivos JSON).

Paso 3: Validación del defecto

El QA Lead revisa el reporte para confirmar que se trata de un defecto válido y no de un error en la configuración del ambiente, un malentendido de la documentación o un error del tester. Esta revisión incluye intentar reproducir el defecto siguiendo los pasos documentados.

Paso 4: Decisión sobre la validez

En este punto se toma una decisión:

- **Si el defecto es válido:** Se procede al paso 5.
- **Si el defecto no es válido:** Se rechaza el reporte, se documenta el motivo del rechazo (por ejemplo, "el comportamiento es el esperado según documentación" o "error del tester") y se cierra el defecto sin acción adicional.

Paso 5: Clasificación del defecto

Para los defectos válidos, el QA Lead asigna dos atributos fundamentales:

- **Severidad:** Impacto técnico del defecto (Alta: impide operación crítica, Media: afecta funcionalidad pero tiene alternativas, Baja: problemas cosméticos o menores).
- **Prioridad:** Urgencia de corrección (Alta: debe corregirse inmediatamente, Media: debe corregirse pronto, Baja: puede esperar).

Paso 6: Registro en el repositorio de defectos

El defecto clasificado se ingresa en el sistema de seguimiento (simulado en hoja de cálculo o tablero) con estado "Abierto", incluyendo toda la información recopilada y la clasificación asignada.

Paso 7: Análisis y plan de corrección

El equipo responsable (simulando el rol de desarrollo) analiza el defecto para determinar su causa raíz y define un plan de corrección, estimando el esfuerzo necesario y el impacto de la solución.

Paso 8: Corrección

Se aplica la corrección necesaria en el ambiente de pruebas. Dado que RESTful-Booker es una API pública que no podemos modificar, en este proyecto académico la "corrección" será simulada o se documentará teóricamente cómo debería corregirse.

Paso 9: Pruebas de verificación (regresión)

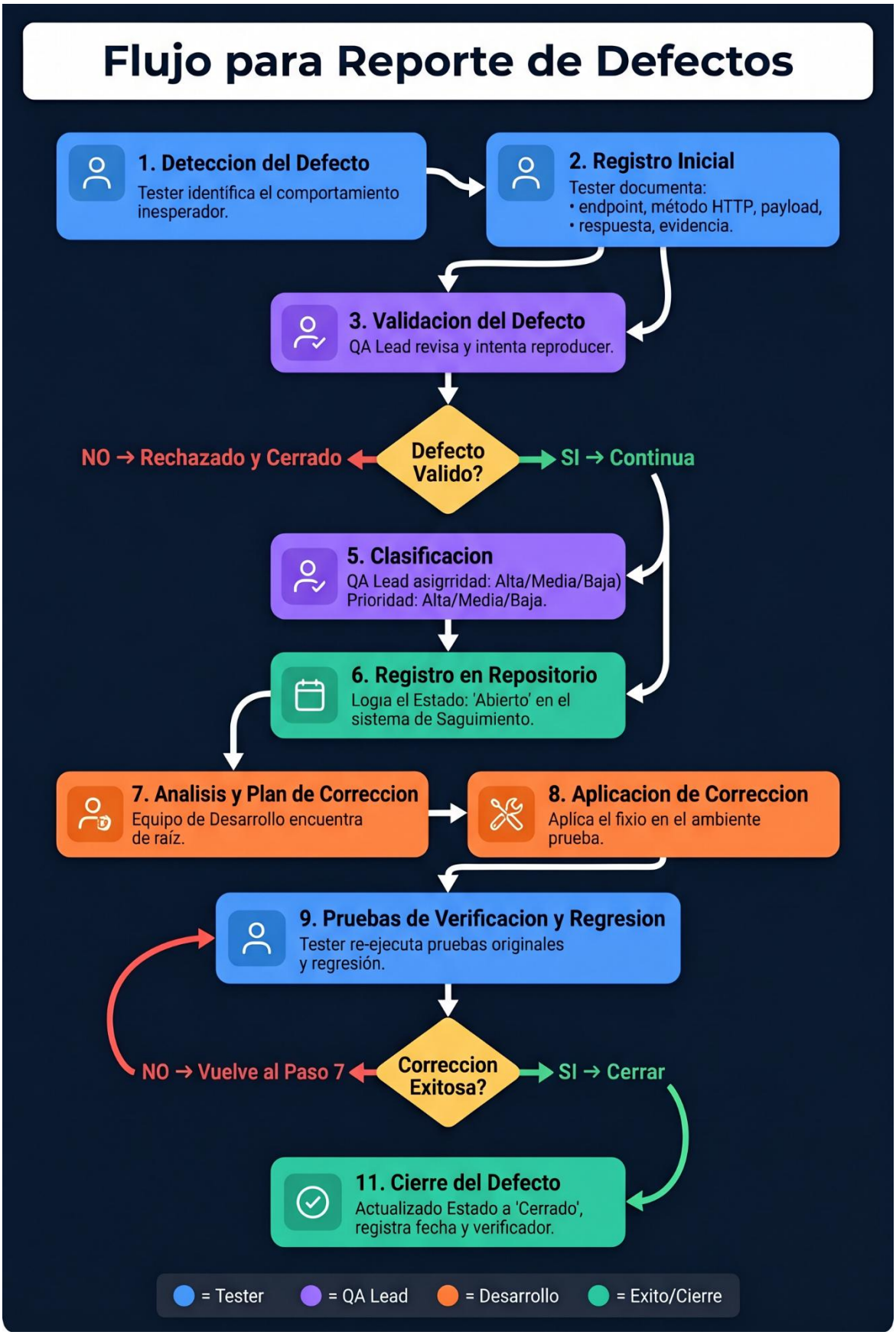
El tester original ejecuta nuevamente los pasos que originalmente causaron el defecto para confirmar que ha sido corregido adecuadamente. También se ejecutan pruebas adicionales para verificar que la corrección no haya introducido nuevos problemas.

Paso 10: Decisión sobre la corrección

- **Si la corrección es exitosa:** El defecto se marca como "Cerrado" y se documenta la solución aplicada.
- **Si la corrección no es exitosa:** El defecto se reabre y se retorna al paso 7 para un nuevo análisis.

Paso 11: Cierre del defecto

Una vez verificado que el defecto ha sido corregido y no hay efectos colaterales, se actualiza el estado a "Cerrado" y se registra la fecha de cierre y el responsable que realizó la verificación.


Flujo para Reporte de Defectos
1. Deteccion del Defecto
Tester identifica el comportamiento inesperador.
2. Registro Inicial
Tester documenta:
• endpoint, método HTTP, payload,
• respuesta, evidencia.
3. Validacion del Defecto
QA Lead revisa y intenta reproducer.
Defecto Valido?
NO → Rechazado y Cerrado
SI → Continua
5. Clasificacion
QA Lead asigrridad: Alta/Media/Baja)
Prioridad: Alta/Media/Baja.
6. Registro en Repositorio
Logıa el Estado: 'Abierto' en el sistema de Saguimiento.
7. Analisis y Plan de Correccion
Equipo de Desarrollo encuentra de raíz.
8. Aplicacion de Correccion
Aplica el fixio en el ambiente prueba.
9. Pruebas de Verificacion y Regresion
Tester re-ejecuta pruebas originales y regresión.
Correccion Exitosa?
NO → Vuelve al Paso 7
SI → Cerrar
11. Cierre del Defecto
Actualizado Estado a 'Cerrado', registra fecha y verificador.
= Tester
= QA Lead
= Desarrollo
= Exito/Cierre

9.2. Formato de reporte de defectos:

Tabla 8 Formato de reporte de defectos

Campo	Descripción
ID del Defecto	DEF-XXX (ej. DEF-001)
Título	Breve descripción del problema
Fecha de Reporte	DD/MM/AAAA
Reportado por	Nombre del tester
Módulo/Endpoint	Ej: /booking (POST)
Tipo de Prueba	Funcional, Negativa, Seguridad, etc.
Severidad	[Alta / Media / Baja]
Prioridad	[Alta / Media / Baja]
Ambiente	URL, versión
Datos de Entrada	Payload enviado, headers
Resultado Esperado	Según documentación
Resultado Obtenido	Respuesta real de la API
Evidencia	Captura de pantalla, archivo JSON
Pasos para Reproducir	Secuencia detallada
Estado	[Abierto / En análisis / Corregido / Cerrado / Rechazado]

Nota: Elaboración propia basada en el proceso de gestión de defectos descrito en prácticas de calidad del software del International Software Testing Qualifications Board.

9.3. Escalamiento de problemas y Criterios de aceptación:

En esta sección se establecen los niveles de escalamiento que se aplicarán cuando se detecten defectos durante la ejecución de las pruebas, así como los criterios cuantitativos que permitirán determinar si el sistema cumple con los niveles de calidad esperados. El escalamiento define quién es responsable de tomar decisiones según la gravedad del problema, mientras que los criterios de aceptación permiten evaluar objetivamente los resultados obtenidos durante el proceso de pruebas.

Tabla 9 Escalamiento de problemas

Escalamiento	Responsable	Criterio de Escalamiento	Acción
Nivel 1	Tester	Defecto menor (cosmético, baja prioridad)	Registrar y continuar pruebas
Nivel 2	QA Lead	Defecto funcional que bloquea pruebas	Evaluar impacto, buscar alternativa temporal
Nivel 3	Product Owner	Defecto crítico (pérdida de datos, seguridad)	Decidir pausa de pruebas o ajuste de alcance

Tabla 10 Criterios de aceptación

Aceptación	Métrica	Valor Aceptable
Tasa de Éxito Funcional	(Casos exitosos / Total casos) * 100	≥ 95%
Cobertura de Endpoints	(Endpoints probados / Total endpoints) * 100	≥ 90%
Densidad de Defectos	Defectos encontrados / Número de endpoints	≤ 2 defectos por endpoint
Tiempo de Respuesta Promedio	Suma tiempos / Número de peticiones	≤ 1000 ms
Gravedad de Defectos	% Defectos críticos vs total	≤ 10% defectos críticos

Nota: Elaboración propia.

10. Gestión del riesgo.

En esta sección se identifican y analizan los principales riesgos que podrían afectar el desarrollo y ejecución del plan de pruebas. Para cada riesgo se evalúa su probabilidad de ocurrencia y el impacto potencial en el proyecto, con el fin de determinar su nivel de criticidad. Además, se definen planes de contingencia y responsables, lo que permite anticipar posibles problemas y establecer acciones preventivas o correctivas para reducir su impacto durante el proceso de pruebas.

Tabla 11 Matriz de Riesgos

ID	Riesgo	Probabilidad (1-5)	Impacto (1-5)	Nivel	Plan de Contingencia	Responsable
R01	La API pública de RESTful-Booker puede estar caída o inaccesible	3	5	Alto	Usar la versión local con Docker como respaldo	Soporte
R02	Limitaciones de tiempo para completar todas las pruebas	4	4	Alto	Priorizar casos de prueba por criticidad; automatizar pruebas repetitivas	QA Lead
R03	Falta de conocimiento en herramientas de API testing	3	3	Medio	Sesiones de capacitación internas; uso de tutoriales; asignación de mentor	Todo el equipo
R04	Cambios en la API durante el período de pruebas	2	4	Medio	Congelar la versión a probar; documentar la versión específica	QA Tester
R05	Pérdida de datos de prueba por reinicio automático (c/10 min)	5	2	Medio	Automatizar creación de datos antes de cada prueba; documentar IDs generados	QA de Automatización

R06	Problemas de conectividad a Internet	2	4	Bajo	Tener la versión local instalada; trabajar offline con Docker	Soporte
R07	Desacuerdo en clasificación de defectos	2	2	Bajo	Sesión de triaje con el equipo; criterios documentados	QA Lead

Nota: Elaboración propia basada en el enfoque de pruebas basadas en riesgos propuesto por el International Software Testing Qualifications Board.

11. Herramientas.

En esta sección se describen las herramientas tecnológicas que serán utilizadas durante el proceso de pruebas para la ejecución, automatización, documentación y seguimiento de defectos. Estas herramientas permiten facilitar la gestión del proceso de testing, mejorar la organización del trabajo del equipo y asegurar la trazabilidad de los resultados obtenidos. La selección de dichas herramientas se basa en su facilidad de uso, disponibilidad y compatibilidad con pruebas de APIs REST.

Tabla 12 Herramientas

Herramienta	Versión	Propósito	Justificación
Postman	v11.0+	Ejecución de pruebas manuales y automatizadas, colecciones de pruebas	Interfaz amigable, soporte para tests en JavaScript, generación de reportes
Newman	v6.0+	Ejecución de colecciones de Postman desde línea de comandos	Permite automatizar y documentar ejecuciones
Insomnia (Opcional)	2023.5+	Respaldo y pruebas exploratorias	Alternativa liviana, útil para pruebas rápidas
Git / GitHub	N/A	Control de versiones de colecciones y scripts	Permite trabajo colaborativo y trazabilidad

Google Sheets / Excel	N/A	Documentación de casos de prueba y métricas	Fácil de usar, permite gráficos y tablas dinámicas
Jira / Trello (simulado)	N/A	Seguimiento de defectos (se puede simular con tablero)	Organización visual del flujo de trabajo
Postman Console	Integrada	Depuración de peticiones y respuestas	Visualización detallada de errores
JSON Validator (online)	N/A	Validación de esquemas JSON	Verificación de estructura de respuestas

Nota: Elaboración propia.

12. Métricas.

En esta sección se definen las métricas que se utilizarán para evaluar el desempeño y la calidad del sistema durante la ejecución del plan de pruebas. Estas métricas permiten medir de forma cuantitativa aspectos como la efectividad de las pruebas, la cobertura funcional, la estabilidad del sistema y la cantidad de defectos detectados. El análisis de estos indicadores facilita la toma de decisiones informadas sobre la calidad de la API y las acciones de mejora necesarias.

12.1. Métricas a evaluar:

Tabla 13 Métricas

ID	Métrica	Fórmula	Unidad	Frecuencia	Objetivo
M01	**Tasa de Éxito Funcional**	(Casos de prueba exitosos / Total casos ejecutados) × 100	Porcentaje (%)	Al finalizar cada ciclo	Medir efectividad de la API
M02	**Cobertura de Endpoints**	(Endpoints probados / Total endpoints identificados) × 100	Porcentaje (%)	Al finalizar planeación	Asegurar amplitud de pruebas
M03	**Densidad de Defectos**	Número total de defectos / Número de endpoints probados	Defectos por endpoint	Al finalizar ejecución	Evaluar calidad por módulo

M04	**Tiempo de Respuesta Promedio**	Suma de tiempos de respuesta / Número de peticiones	Milisegundos (ms)	Durante ejecución	Evaluar rendimiento
M05	**Distribución de Defectos por Severidad**	(Defectos de severidad X / Total defectos) × 100	Porcentaje (%)	Al finalizar ejecución	Priorizar correcciones
M06	**Tasa de Detección de Defectos**	Defectos encontrados en pruebas / Total defectos (estimado)	Razón	Al finalizar	Efectividad del proceso
M07	**% de Pruebas Automatizadas**	(Casos automatizados / Total casos) × 100	Porcentaje (%)	Al finalizar	Medir avance en automatización
M08	**Estabilidad de la API**	(Peticiones exitosas / Total peticiones) × 100	Porcentaje (%)	Durante ejecución	Confiabilidad del sistema

Nota: Elaboración propia basada en métricas de calidad de software utilizadas en aseguramiento de calidad y documentadas por el International Software Testing Qualifications Board.

12.2. Plantilla:

Tabla 14 Plantilla

Con el fin de facilitar el seguimiento y análisis de los resultados obtenidos durante la ejecución de las pruebas, se establece una plantilla para registrar las métricas por cada endpoint evaluado

Endpoint	Método	# Pruebas	# Exitosas	# Fallidas	Tiempo Prom (Ms)	Defectos encontrados
/AUTH	POST					
/BOOKING	GET					
/BOOKING	POST					
/BOOKING/{ID}	GET					
/BOOKING/{ID}	PUT					
/BOOKING/{ID}	PATCH					
/BOOKING/{ID}	DELETE					
TOTAL						

Nota: Elaboración propia.

13. Referencias

RESTful-Booker. (2024). Documentación oficial de RESTful-Booker. https://restful-booker.herokuapp.com/

Organización Internacional de Normalización. (2011). ISO/IEC 25010:2011: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. https://www.iso.org/standard/35733.html

Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., & Berners-Lee, T. (1999). Hypertext transfer protocol — HTTP/1.1 (RFC 2616). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2616

Microsoft. (2023). Microsoft REST API guidelines. https://github.com/microsoft/api-guidelines

Institute of Electrical and Electronics Engineers (IEEE). (2008). IEEE standard for software and system test documentation (IEEE Std 829-2008). https://standards.ieee.org

International Software Testing Qualifications Board. (2023). ISTQB Certified Tester Foundation Level Syllabus. https://www.istqb.org

Institute of Electrical and Electronics Engineers. (2008). IEEE Standard for Software Test Documentation (IEEE 829). https://standards.ieee.org

International Organization for Standardization. (2011). ISO/IEC 25010 Software Quality Model. https://www.iso.org